function *f*:

l = f(k)

The performance of the implementation is determined by the choice of function f and the method of handling collisions.

Hashing Function.

A good hashing function will produce a uniform distribution of locations in a hash table given a random set of keys. There have been many suggestions for the implementation of the hashing function. The purpose of the hashing function in the case of the meta-assembler is to take an alphabetic key (either the mnemonic name or symbol name) and return an address within the array at which the data on the key will be stored. One of the simplest and most effective hashing algorithm is the division hashing algorithm. Characters in the key are multiplied by their positions in the string and added together. The total is then taken modulo the table size to give the position in the table.

h(K) = K MOD M where

K is the key to be hashed,
h(K) is the hashed key,
M is the size of the table.

Hashing Collisions.

The main drawback to hash tables is that collisions occur where two or more keys 'hash' to the same location in the table. Figure 30 shows how the performance of a hash table is drastically reduced when it becomes more than 80% full. There have been a number of proposals to the solution of this problem, some of which are:

* Linear Probing. When a collision occurs, one of the colliding data elements

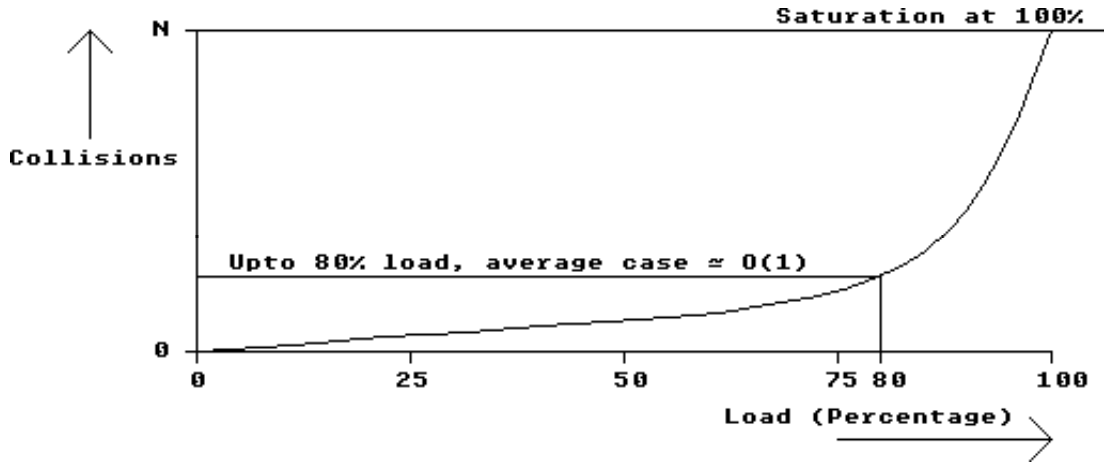

Figure 30. *Affect of Loading on the Performance of a Hash Table*

is put in the next available location in the table.

* Nonlinear Probing. A secondary hashing function is applied to any key which collides when hashed by the primary function.

* Bucket Hashing. Each location in the hash table can contain many data items. The data structure which provides this 'bucket' can be:

- Linked list.
- Binary search tree.
- Another hash table.

Operation On Table	Best Case	Worst Case
Insert	O(1)	O(1)
Retrieve	O(1)	O(Log(N))
Delete	O(1)	O(N)
Report	$O(N^2)$	$O(N^3)$

Figure 31. *Performance of a Hash Table with Binary Search Tree 'Buckets'.*

The method selected for the implementation is bucket hashing because it is less complicated than the other methods and can be dynamic which allows a hash table of any size (which stops the performance of the table from dropping drastically). In terms of overall performance, it is better to use a binary search tree when bucket hashing because they are quicker than linked lists (see figure 29) but are still dynamic (which hash tables are not).

Figure 31 gives the big O performance of the hashing table and binary search tree buckets combination. Performance on all functions is good except the report function where performance drops to $O(N^3)$. This, is still acceptable, however, as the operation is only used when symbol tables are being dumped (i.e., at most once in an assembly run). The other possibility would be to report without alphabetical sorting of the key items which would reduce big O value.

The three modules which implement the hash table are:

Table. This module implements the hash table as an abstract state machine. Operations on the table use the BST (Binary Search Tree) object which is implemented by the TableTrees module. All operations in this module are completely data independent (except that the key must be a string of characters).

TableTrees. This module implements a BST abstract data type and associated operations. The BST is exported opaquely and is therefore totally secure.

TableExt. This module (Table Extensions) contains the data structure which defines the data to be stored in the table and the operations on this data structure which are data-dependent. As two complete sets of data are required (one for instruction mnemonics and assembler directives, and one for symbols) the data structure used is a variant record. These allow for the most efficient storage of data. The data to be stored is:

For mnemonics and assembler directives:

* Name (Key).
* Type (or class).
* Machine code mask.

For symbols:

* Name (Key).
* Value.
* Status (Absolute or Relative).

This module provides two operations, **InsertCommand** and **InsertSymbol** which allow for the convenient insertion of either commands or symbols into the table. The advantage of using these operations is that a data structure does not need to be constructed to use them. They construct the appropriate data structure and then use the data structure-independent operation **Insert** (in the Table module) to enter them into the table. Operations are also supplied to write and read the data structure from and to a file, although these are not used by the meta-assembler. The other two operations provided allow the conversion of a machine instruction mask from and to a readable format (again, these are not used).

5.4.11. Assembler Definition Module (ADM).

The purpose of the assembler definition module is to encompass all the assembler dependent data structures, parameters and algorithms. The two parts of the ADM, the definition and implementation modules, serve two purposes.

Definition Module.

This module describes the assembler to the rest of the modules via a number of parameters (Modula-2 constants). These are:

WORDLENGTH. The wordlength describes the length of a basic machine code instruction unit in terms of the number of bits required. Although the decision as to what constitutes a word in any processor is somewhat subjective, virtually all processors have an easily identifiable word length. Figure 32 gives a number of processors and their associated word length.

Processor	Wordlength (in bits)
Intel 8088	8
Motorola 6800	8
Motorola 68000	16
Commodore 6510	8
Zilog Z80	8

Figure 32. *Wordlengths of Various Processors.*

MAXINDICES. This constant describes the maximum number of indices allowable in an operand.

MAXOPERANDS. This constant describes the maximum number of operands allowable in an assembly language statement.

MAXWORDSPERINSTRUCTION. This constant describes the number of machine words in the longest machine instruction.

MEMLOW, MEMHIGH. These two constants describe the range which the processor can address. Depending on whether the assembler is specific to one computer, this can also describe the maximum memory physically possible on that computer.

NOOFREGISTERS. This constant is used to define an array which must be filled with the character string representations of all the registers that can be used in the assembly language being described. The array, ValidRegisters, is filled within the main body of the ADM implementation module.

Data structures to describe a machine word, instruction, all valid addressing modes, an operand element and a decoded line are included in the ADM definition module for convenience.

Implementation Module.

The ADM implementation module describes two operations which together form

the algorithmic part of the specification for a processor.

InsertOpcodesInTable. This operation, as its name suggests, inserts all the assembler opcodes into the opcode/symbol table. This operation is called from the coordination (M2Assem) module. The operation uses the **InsertCommand** operation exported by the TableExt module to insert opcode manes into the table with their associated instruction mask and type. The type of the instruction is dependent on the layout of the instruction formed when using the opcode and is chosen by the programmer. It is used within the **Assemble** operation to select the appropriate algorithm to assembler the instruction.

Assemble. This operation forms the main part of the ADM implementation module and implements the code generator of a traditional dedicated assembler. The operation has an overall structure which is assembler-independent. This is shown in pseudo-code in figure 33. Each **Type** operation assembles a different format of machine instruction. As was shown in section 4.5., the regularity of a processors instruction set determines the number of instruction formats (and therefore **Type** operations) which are required. It follow that the length and complexity of the **Assemble** instruction depends to a great extent on the regularity of the instruction set of the processor being described.

The **Assemble** operation, and therefore each **Type** operation, is supplied with two parameters:

1. The data structure containing the decoded assembly language statement which is produced by the **ScanLine** operation in the lexical scanner module (Lex).
2. The current pass number (each **Type** operation used will be called twice, once for each pass).

The **Assemble** operation must produce two data items:

1. The machine code instruction corresponding to the decoded assembly language statement input.
2. The length of the above instruction.

```
operation Assemble

  operation Type01
  operation Type02
  :
  :
  operation TypeN

begin
  if PassOne then
    CheckForLabel
    if IsLabel then
      InsertLabelInTable
    end
  end
  if IsCommand then
    case CommandType of
      1: InstructionLength := Type01
      2: InstructionLength := Type02
      :
      :
      N: InstructionLength := TypeN
    else
      AssemblePseudoOp
    end
  else
    NoCommandError
  end
end
```

Figure 33. *Internal Structure of Assemble Operation.*

The task which the **Type** operations must perform is dependent on which pass the meta-assembler is currently on.

In pass one the sole task is to determine the length of the machine code instruction. Depending on the processor being assembled for, this may simply involve returning the appropriate value (in which case pass one operation become trivial). However, with some processors the instruction is not constant and depends on the data length (modifier) and operands. In this case, more work is done which sometimes involves analysing operands.

In pass two, the **Type** operations perform the translation of decoded assembly language statement into machine language instruction. This process generally

requires four steps (which may be in different orders or with some steps unnecessary).

1. The addressing mode of each of the operands in the assembly language statement must be determined (using the **ScanOperand** operation exported by the lexical scanner).
2. The number of operands and the correct addressing modes must be checked. If there is an error an exception must be raised with a suitable error message.
3. Each operand and index must be evaluated (using the **Evaluate** operation exported by the Expression module) or alternatively checked to see whether it is the appropriate register (using the **CheckRegister** operation, also exported from Expression). If an expression is not in the required range or a register is not of the correct type an exception should be raised.
4. Finally, the machine code instruction should be assembled. This may involve the conversion of operands into the appropriate format required.

Once the machine code instruction has been assembled, it can be returned together with its length. Control then passes back to the calling module (M2Assem in this case) for assembly to continue.

In order to test this design an ADM was developed for the Motorola MC68000 processor. A description of this design follows.

5.4.12. ADM for the Motorola MC68000.

The first task required in producing an ADM for the MC68000 was to provide values for the processor parameters.

WORDLENGTH = 16
MAXINDICES = 2
MAXOPERANDS = 2
MAXWORDSPERINSTRUCTION = 5
MEMLOW = 0
MEMHIGH = 1000 X 1024 (1 MByte)

NOOFREGISTERS = 17

The next task, turning to the implementation module, was to define all the possible registers. This was done by making assignments to the ValidRegisters array in the module body.

```
ValidRegisters[1] := "D0";
          :
          :
ValidRegisters[8] := "D7";
ValidRegisters[9] := "A0";
          :
          :
ValidRegisters[16] := "A7";
ValidRegisters[17] := "SP"
```

The **InsertOpcodesInTable** operation then needed to be filled with the appropriate **InsertCommand** operations. The problem is that the IEEE defined opcode set does not match one for one the Motorola defined opcode set, meaning that one IEEE opcode can map onto several Motorola opcodes. The solution is to have a number of pseudo-Type operations which select the appropriate instruction format by examining the addressing modes of operands. Figure 34 shows all the IEEE standard mnemonics, the appropriate opcode mask, the type of the opcode and the corresponding Motorola opcode mnemonic(s). Note that the instruction types, as well as the standard 18, include an extra eight which correspond to IEEE standard opcodes which map onto several formats of instruction. The InsertCommand operations were then written from this table.

The other mapping which took place was from the IEEE standard addressing modes to the appropriate Motorola addressing modes. This can be seen in figure 35.

An operation, **RegisterType**, was is used to determine whether a register is a data or address register. Another operation allows the addition of a binary word at any position in another word. Many of the Motorola instruction formats include a field for a register and an effective address (see figure 6). Two

IEEE STD Mnemonic	Machine Code Mask MSB 15	LSB 0	Class	Motorola Mnemonic
ADD	1101000000000000		19	ADD,ADDA,ADDQ,ADDI
ADDD	1100000100000000		6	ABCD
ADDC	1101000100000000		5	ADDX
SUB	1001000000000000		19	SUB,SUBA,SUBQ,SUBI
SUBD	1000000100000000		6	SBCD
SUBC	1001000100000000		5	SUBX
MULU	1100000011000000		3	MULU
MUL	1100000111000000		3	MULS
DIVU	1000000011000000		3	DIVU
DIV	1000000111000000		3	DIVS
CMP	1011000000000000		19	CMP,CMPA,CMPI,CMPM
NEG	0100010000000000		13	NEG
NEGC	0100000000000000		13	NEGX
NEGD	0100100000000000		15	NBCD
EXT	0100100000000000		17	EXT
DBR	0101000011001000		12	DBRA,DBF
DBE	0101011111001000		12	DBEQ
DBNE	0101011011001000		12	DBNE
DBC	0101010111001000		12	DBCS
DBNC	0101010011001000		12	DBCC
DBP	0101101011001000		12	DBPL
DBN	0101101111001000		12	DBMI
DBNV	0101100111001000		12	DBVS
DBGT	0101111011001000		12	DBGT
DBGE	0101110011001000		12	DBGE
DBLT	0101110111001000		12	DBLT
DBLE	0101111111001000		12	DBLE
DBH	0101001011001000		12	DBHI
DBNH	0101001111001000		12	DBLS
AND	1100000000000000		19	AND,ANDI
OR	1000000000000000		19	OR,ORI
XOR	1011000000000000		19	EOR,EORI
NOT	0100011000000000		13	NOT
SHR	1110000000001000		21	LSR
SHL	1110000100001000		21	LSL
SHRA	1110000000000000		21	ASR
SHLA	1110000100000000		21	ASL
ROR	1110000000011000		21	ROR
ROL	1110000100011000		21	ROL
RORC	1110000000010000		21	ROXR
ROLC	1110000100010000		21	ROXL
TEST	0100101000000000		13	TST
TEST1	0000000100000000		22	BTST
TESTSET	0000000111000000		22	BSET
TESTCLR	0000000110000000		22	BCLR
TESTNOT	0000000101000000		22	BCHG

Figure 34. *IEEE Standard Opcodes with Motorola Equivalents (1 of 2).*

CHK	0100000110000000	3	CHK
LD	0000000000000000	23	LEA,MOVE,MOVEA, MOVEP,MOVEQ
LDM	0100110010000000	14	MOVEM
ST	0000000000000000	23	MOVE,MOVEP
STM	0100100010000000	14	MOVEM
MOV	0000000000000000	23	MOVE,MOVEM,MOVEP, MOVEA,MOVEQ
XCH	1100000100000000	24	EXG,SWAP
CLR	0100001000000000	13	CLR
SET	0101000011000000	10	ST
SETE	0101011111000000	10	SEQ
SETNE	0101011011000000	10	SNE
SETC	0101010111000000	10	SCS
SETNC	0101010011000000	10	SCC
SETP	0101101011000000	10	SPL
SETN	0101101111000000	10	SMI
SETV	0101100111000000	10	SVS
SETNV	0101100011000000	10	SVC
SETGT	0101111011000000	10	SGT
SETGE	0101110011000000	10	SGE
SETLT	0101110111000000	10	SLT
SETLE	0101111111000000	10	SLE
SETH	0101001011000000	10	SHI
SETNH	0101001111000000	10	SLS
BR	0110000000000000	25	BRA,JMP
BE	0110011100000000	11	BEQ
BNE	0110011000000000	11	BNE
BC	0110010100000000	11	BCS
BNC	0110010000000000	11	BCC
BP	0110101000000000	11	BPL
BV	0110100100000000	11	BVS
BNV	0110100000000000	11	BVC
BGT	0110111000000000	11	BGT
BGE	0110110000000000	11	BGE
BLT	0110110100000000	11	BLT
BLE	0110111100000000	11	BLE
BH	0110001000000000	11	BHI
BNH	0110001100000000	11	BLS
CALL	0110000100000000	25	BSR,JSR
RET	0100111001110101	18	RTS
RETR	0100111001110111	18	RTR
RETE	0100111001110011	18	RTE
NOP	0100111001110001	18	NOP
PUSH	0100100001000000	26	LINK,PEA,MOVE
POP	0100000111000000	26	UNLK,MOVE
WAIT	0100111001110010	18	STOP
BRK	0100111001000000	16	TRAP
BRKV	0100111001110110	18	TRAPV
RESET	0100111001110000	18	RESET

Figure 34. *IEEE Standard Opcodes with Motorola Equivalents (2 of 2).*

Addressing Mode Equivalents:

Motorola Format	IEEE Format
Register Direct Dn,An	Direct .Dn,.An
Address Register Indirect (An)	Indirect @.An
Addr. Reg. Indirect with Post-increment (An)+	Auto-post-increment .An+
Addr. Reg. Indirect with Pre-decrement -(An)	Auto-pre-decrement -.An
Addr. Reg. Indirect with Displacement d16(An) or (d16,An)	Indirect-post-indexed @.An(d16)
Addr. Reg. Indirect with Index d8(An,Xn.W) or (d8,An,Xn.W) d8(An,Xn.L) or (d8,An,Xn.L)	Indirect-post-indexed @.An(d8,.Xn)
Absolute Long Address xxx.L	Absolute /xxx
Absolute Short Address xxx.W	Absolute /xxx
Program Counter with Displacement d16(PC) or (d16,PC)	Indirect-post-indexed @*(d16)
Program Counter with Index d8(PC,Xn.W) or (d8,PC,Xn.W) d8(PC,Xn.L) or (d8,PC,Xn.L)	Auto-post-index @*(d8,.Xn)
Immediate Data #xxx or #<data>	Immediate #xxx or #<data>

Figure 35. *IEEE Addressing Modes and their Motorola Equivalents.*

operations were therefore provided to add a register and an effective address at any point within a machine instruction word.

The MC68000 is one of the processors for which the calculation of the length of an instruction is not always determined by its opcode. Pass one of the assembler is therefore more complicated in some **Type** operations.

For those instructions whose length is not always one word, the length is determined by the effective address field. Depending on the addressing mode, an instruction can be from one to five words long. The **CalcEA** operation determines the length of these instruction on pass one and assembles the effective address field on pass two.

The checking for the correct addressing mode, number of operands and number of indices is done within each **Type** operation, where necessary. The command modifier is also checked to determine whether an assembly language statement is to operate on byte, word or longword quantities.

The pseudo-**Type** operations use the **ScanOperand** operation to determine the addressing modes of operands which in turn determines which Motorola instruction format to choose. Calls to other **Type** operations occur within these operations.

The complete listing of the example ADM for the MC68000 is included in the appendix.

5.4.13. Coordination Module (M2Assem).

The coordination module drives the rest of the meta-assembler modules. Its structure is very similar to that of a dedicated two-pass assembler. The module contains two operations, **PassOne** and **PassTwo** which control each pass of the meta-assembler. The module opens and closes all the files used during assembly (input source program, output listing and object code files) by calls to the appropriate operations in the Interface, Listing and ObjectGenerator modules. The command line format which operates the meta-assembler is very simple:

M2Assem <filename>

The filename in the input file does not contain any extension. Default extensions are added to this file name to obtain the complete names of each file:

<filename>.asm input assembly language program.

```
<filename>.obj     output object code file.
<filename>.lst     output listing file.
```

5.4.14. Assembler Directives Module (PseudoOps).

This module implements the assembler-independent assembler directives described in section 5.3.3. The directives have associated Type values above 1000. When the **Assemble** operation in the ADM cannot match the type to one of its **Type** operations it calls an operation in the PseudoOps module called **AssemblePseudoOp** which performs the same function as **Assemble**, except that it selects between the possible assembler directives.

5.5. Implementation.

Following the object oriented development philosophy, modules where implemented individually (as far as possible). The order in which actual implementation of modules took place was determined to some extent by the way in which modules coupled with each other. For instance, it was necessary to implement the Strings, Exceptions and Table handling modules first so that testing of other modules could take place when they were developed.

5.6. Testing.

There were two levels of testing that were carried out during the development of the meta-assembler:

1. Individual Module Testing.
2. Module Integration Testing.

The first level of testing was carried out after the implementation of each module to ensure that the module behaved as expected. In order to carry out this testing three test modules were developed.

1. TestStrings. This module was used to test the operations exported from the Strings module to ensure that they were functionally correct.
2. TestLex. This module was used to test the functionality of the **ScanLine** operation exported from the lexical scanner module.
3. TestTable. This module was originally developed to test the three table handling modules: Table, TableTrees and TableExt. However, because the modules needed the table facilities in order to be tested properly, this module was developed further to incorporate tests for other modules as they were developed. The TestLex module's features were incorporated into this module later on so that a DecodedLine structure could be obtained.

The module presents the tester with a menu of possible operations:

1. A new symbol or opcode can be added to the table.
2. A symbol or command can be retrieved from the table.
3. A symbol or command can be deleted from the table.
4. An assembly language statement can be assembled. This must previously have been decoded by the lexical scanner.
5. The contents of the table can be printed out in order.
6. The number of entries in each hash location can be displayed.
7. An assembly language statement can be entered and then decoded by the lexical scanner.
8. An expression can be entered (which may contain previously defined labels) and evaluated by the expression evaluator.
9. The default base of constants can be changed.
10. A listing file can be generated by entering an assembly language statement, its equivalent machine code and the number of times this is to be repeated.

This last test module therefore performs both individual module testing and to some extent integration testing. The real integration testing could only begin after the implementation of the coordination module, M2Assem. It is interesting to note that this was one of the last modules to be implemented.

Testing of individual modules using the three test programs actively involved the following test principles:

Internal Testing.

This is a process of comparing the design of modules and the way in which the code implements the design.

Complexity analysis - Examine complex areas of code to determine whether they are necessary or the result of bad design.

Structure analysis - Are all parts of the code called from all possible conditions? Are there any unnecessary conditions which are never executed? Do program structures have a single entry and exit point?

Internal testing was to some extent not so important as the program design and implementation was written by the same person. However, when using unsuitable methods of implementation it allowed the identification and correction of these areas.

External Testing.

This is involved in testing the implementation by regarding it as a box which should produce the correct output when given suitable input. The test process can be summarised:

1. Identify test data for the system from the specification.
2. Specify the expected results for each test case.
3. Run the program and record actual results.
4. Compare expected and actual results.

The design of the test data determined how well the implementation could be tested. The following principles were used.

Equivalence Partitioning.

'The object of equivalence partitioning is to identify from the logic of the specification (program design) the categories into which input values are grouped'.

Boundary Case Analysis.

'An extension of equivalence partitioning which concentrates on the edges of the class'.

The testing both at individual and integration levels went well. The integration testing did highlight some areas of the design which, although not wrong, could definitely be improved (see Conclusion).

6. CONCLUSION.

6.1. Usefulness of Meta-Assemblers.

The usefulness of meta-assemblers has been proven by past implementations. Their use has evolved from their macro-based implementations in the 1960s which were used as a forerunner to the high level language, through the 1970s and 1980s when the development of a multitude of microprocessors lead to the need for a tool to develop and implement assemblers quickly and easily, right up to the present day. It is clear that the continuing development of general and special purpose microprocessors will require such a tool for the foreseeable future.

Another area in which the meta-assembler will be used in the future is that of the Universal Compiler. Such a tool can compile a large number of high level languages and produce object code for a large number of computers. The final output stage of this tool is always some form of meta-assembler. The development of universal compilers will continue for the foreseeable future, ensuring the continuing development of faster and more flexible meta-assembler. Such universal compilers have already been implemented, one such being the Amsterdam Compiler Kit which is described in Tanenbaum's 'Structured Computer Organization'[13].

The construction of meta-assemblers will continue to reflect the changes affecting high level language design. As more and more software is being designed using Object Oriented Techniques, so the construction of meta-assemblers will reflect the use of these techniques. The trend so far has been to turn away from table-driven designs in favour of meta-assemblers with a program-based assembler specification. In terms of current programming trends, these will eventually take the form of semi-intelligent, object oriented specifications written in a suitably adapted 5th generation language.

6.2. Evaluation of Implementation.

6.2.1. Overview.

The sample implementation takes a data and algorithm-driven approach to the meta-assembler problem. By using the principles of Object Oriented Programming it was possible to re-design a traditional two-pass dedicated assembler into its assembler-dependent and assembler-independent parts. This enabled the implementation of an assembler in Modula-2 which can be re-configured easily to assemble for another processor.

It was decided that the syntax of the assembly language which can be entered into the implementation should conform to the IEEE Assembly Language Standard. In the context of the use of meta-assemblers, this provides a consistent format to which all assembly languages can conform.

6.2.2. Speed Of Assembly.

The speed of the meta-assembler implementation compares well with other implementations. Running on an IBM XT compatible computer at 10 MHz, assembly speed for various length test programs ranged from 370 lines/minute to 720 lines/minute. In direct comparison with the Generic Meta-Assembler described earlier in section 4.7.7., assembly speed was slightly faster when assembling a 200 line program at 4.77 MHz (3.52 lines/second for the sample implementation compared with 2.74 for the Generative Meta-Assembler).

6.2.3. Development Time for a New ADM.

Unfortunately, there was not enough time left to implement any other ADM modules (apart from the MC68000 ADM described). A design for a 6502 ADM was developed but not implemented.

The structure of the ADM and its function suggest that development time for processors with orthogonal instruction set should be considerably less than for those whose instruction sets that are not orthogonal. This was certainly true when developing the 6502 ADM as opposed to the MC68000 ADM.

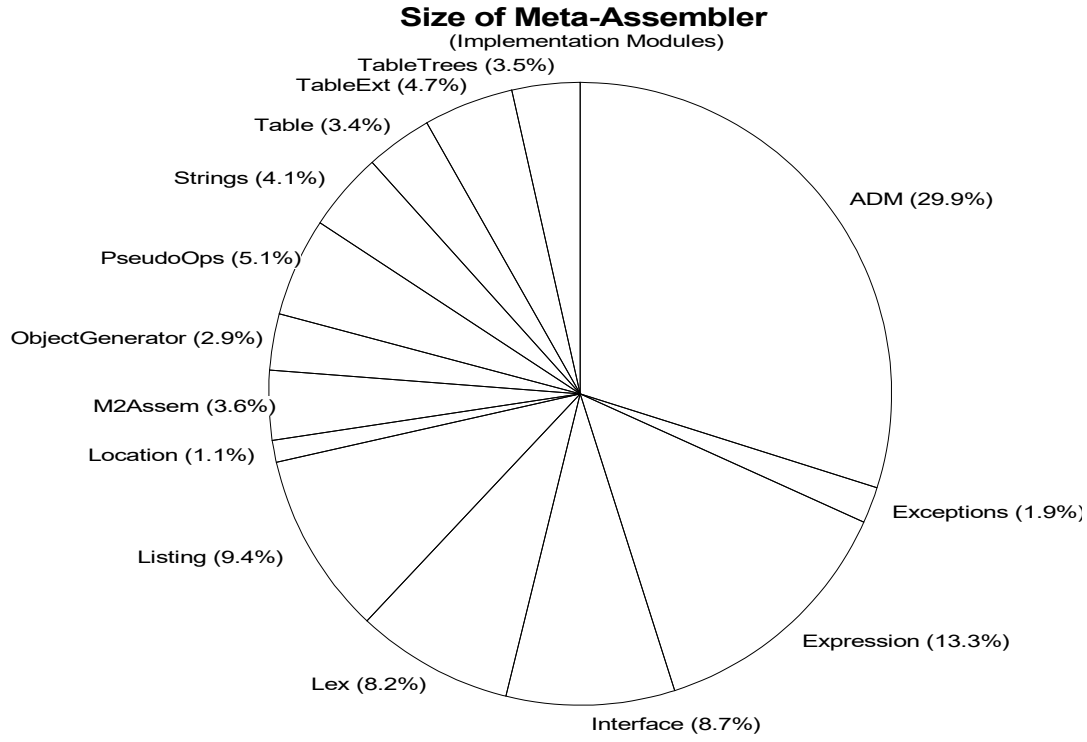

Figure 36.

Figure 36 shows how much of the complete program the separate modules of the implementation constitute. It can be seen that the MC68000 ADM constitutes 30% of the complete meta-assembler source program. It is felt, however, that a significant improvement on the length of this file could be made given more time (see section 6.2.4.)

The physical time spent developing the ADM module was again about 30% of the development time of the complete meta-assembler. This translates into approximately 30 man/hours, which compared with other meta-assembler implementations discussed earlier is very reasonable, given the fact that the MC68000 does not have a particularly orthogonal instruction set.

6.2.4. Immediate Improvements.

This section contains a number of suggestions of what would be done to improve the sample implementation given more time. Each suggestion is followed by either a 'T' or a 'D' which signifies whether the improvement is simply due to a lack of time or whether the improvement constitutes a change in the design of the meta-assembler.

1. The Table module should deallocate memory used (which currently it does not). While this is not a problem for a version implemented on a single-task computer (such as an IBM XT compatible running MS-DOS), this would be a problem on a multi-tasking computer (such as the Commodore Amiga) where memory deallocation is important. (T).

2. A serious flaw in the design of the sample implementation is that the valid addressing modes for any instruction are not included in the table with the opcodes. This leads to a very much larger ADM implementation module. The design should be changed, therefore, so that a set of user defined addressing modes (corresponding to those implemented on the target processor) should be included with the opcodes in the table for every operand used in an instruction. (D).

3. Only one object code format has been implemented. The number should be increased to at least three:

* Generic.
* Motorola-S format.
* Intel format.

The selection of the format should be included in an assembler directive so that it can be selected at will by the programmer. (T/D).

4. The design of the ADM for the Motorola MC68000 is not very efficient and could be improved greatly (partly because of point 2 above). (T/D).

5. More ADMs should be written for other processors. (T).

6. No facilities are provided within the Location module to handle page/segment addressed systems. So, for example, a proper Intel 80x86 assembler could not be implemented at present. This problem stems from the lack of

information provided in the IEEE standard about how page addressed systems should be handled. A parameter (constant) could be included in the ADM module which would inform the meta-assembler which type of addressing system is to be used (either linear or paged). (D).

7. The speed of implementation could be improved by using an execution profiler to determine where the most time is being spent. Such a profiler is included in the JPI Topspeed Modula-2 package. Very time-critical sections could be re-written using assembly language but this promotes non-portability and should therefore be used as little as possible. (T).

8. The expression evaluator should be improved to allow expressions with more than two operands and priorities provided by parentheses. Such an evaluator would probably be stack-based. The re-implementation of the expression evaluator should not affect the structure of any of the other modules. (D).

6.2.5. Future Developments.

This section contains suggestions for future developments which would require large alterations to the current implementation.

1. The meta-assembler could be re-designed so that it can produce relocatable code and provide more information to a linker such as external references and symbolic debugging information.

2. A meta-linker would be a useful and viable tool to the meta-assembler. This would produce object code formats based on a specification which is either included within the ADM or separately. If this was included within the ADM then the ADM would specify the complete translation process from an assembly language program to an object code program.

3. The IEEE standard was not necessarily the best standardised processor description method to use. Several methods have been implemented which address some of the problems caused by the IEEE standard. These include

the fact that large lexical scanners are needed and that some aspects (such as paged addressing systems) are omitted. The reader is referred to references 27 and 28 for more information.

4. Assembler facilities could be improved to make the meta-assembler more powerful. These would include macro facilities and block structuring of code. This would bring the syntax of the assembly language more in line with those of modern dedicated assemblers.

7. BIBLIOGRAPHY.

Books:

1. Gorsline, George W. Assembly and Assemblers. Englewood Cliffs: Prentice Hall, 1988, 355-410.

2. Barron, D.W. Assemblers and Loaders, third edition. London: Macdonald and Jane's, 1978.

3. Calingaert, Peter. Assemblers, Compilers and Program Translators, Potomac, Maryland: Pitman, 1979.

4. Berry, R.E. Programming Language Translation. Chichester: Ellis Horwood, 1982.

5. Graham, Robert M. Principles of System Programming. New York: John Wiley & Sons, 1975, 49-80.

6. Leventhal, Lance A. Z80 Assembly Language Programming. Berkeley, California: Adam Osborne & Associates, 1979.

7. Motorola. MC68000 8-/16-/32-Bit Microprocessors Programmer's Reference Manual, fifth edition. Portsmouth: Grosvenor Press, 1988.

8. Woodman, M., Griffiths, R., Souter, J., Davies, M. Portable Modula-2 Programming. Maidenhead: McGraw Hill, 1989.

9. Meyer, Bertrand. Object Oriented Software Construction. Hemel Hempstead: Prentice Hall, 1988.

10. Gear, C.W. Computer Organization and Programming, fourth edition. Singapore: McGraw Hill, 1985, 337-370.

11. Feldman, Michael B. Data Structures with Modula-2. Englewood Cliffs: Prentice Hall, 1988.

12. Hewitt, J.A., Frank, J.R. Software Engineering in Modula-2. Basingstoke: Macmillan, 1989.

13. Tanenbaum, Andrew S. Structured Computer Organization, second edition. Englewood Cliffs: Prentice Hall, 1984. 181-212, 351-366.

14. Watson, Desmond. High-Level Languages and Their Compilers. Reading, Massachusetts: Addison Wesley, 1989. 98-101.

Articles:

15. Wing, Kam., Ping Lam, Chi. MAUFI - A Meta Assembler with a User-Friendly Interface. ACM Sigsmall Symposium on Small Systems, May 1985, 57-63.

16. Tiberghien, J., Teaymans, J. A Cross-Assembler Suitable For Most Of The 8-Bit Micros. Euromicro-Newsletter (France), 1977, Vol 3, Part 2, 80-83.

17. Fujita, S., Yoshida, K. A General Purpose Cross-Assembler for Microprocessors. Euromicro-Newsletter (France), 1977, Vol 3, Part 4, 88-89.

18. Leibson, Steven H. Software Tools to Handle All Microprocessor Traffic. EDN, 8 June 1989, 89-97.

19. Heath, J.R., Patel, S.M. How to Write a Universal Cross-Assembler. IEEE Micro, August 1981, 45-66.

20. Distler, R.J., Shaver, M.A. Trial Implementation Reveals Errors in IEEE Standard. Computer, July 1982, 76-77.

21. Wulf, William A. Compilers and Computer Architecture. COMPUTER, July 1981, 41-47.

22. Wilburn, D.L., Schleimer, S. Step Development Tools: METASTEP Language System. Communications of the ACM, 1985, Vol 12, 157-164.

23. Anacona, M. A High Level Language Based System for Cross-Assembler Definition. Software - Practice & Experience, December 1985, Vol 15, 1159-1184.

24. Skordalakis, E. Meta-Assemblers. IEEE Micro, April 1983, 6-16.

25. Fisher, W. Microprocessors Assembler Language Draft Standard. COMPUTER, July 1981, 41-47.

26. Ferguson, D. The Evolution of the Meta-Assembler Program. Communications of the ACM, 1966, Vol 9, 190-196.

27. Kott, R.K., Ambroziak, K. The Cross-Assembler Universality Versus the Standardization of Microcomputer Assembly Languages. Euro-Micro 81, 1981, 209-217.

28. Anacona, M., Clematis, A., Dodero, G., Pino, A. Experiences with the use of a Universal Cross-Assembler Based on the IEEE Proposed Standard. MICROSYSTEMS: Architecture, Integration & Use, 1982, 55-64.

29. Anderson, Brian R. Modula-2: A 68000 Cross-Assembler - Part 1. Dr Dobb's Journal of Software Tools, April 1986, Vol 11, Issue 4, 52-56, 76-81.

30. Anderson, Brian R. Modula-2: A 68000 Cross-Assembler - Part 2. Dr Dobb's Journal of Software Tools, May 1986, Vol 11, Issue 5, 44-57, 80-92.

31. Anderson, Brian R. Modula-2: A 68000 Cross-Assembler - Part 3. Dr Dobb's Journal of Software Tools, June 1986, Vol 11, Issue 6, 46-62.

32. Chui, Peter P. K., Fu, Sammy T.K. A Generative Approach to Universal Cross Assembler Design. SIPLAN Notices, January 1990, Vol 25, No 1, 43-51.